

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: COMPILER DESIGN

COURSE CODE: CSA14

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool | Weightage: 20%

QUESTIONS: 5

Duration :60 Mins

Total Marks:50

Assessment Mode: In Class

Annexure A

Scenario Based Task

Code of Conduct:

*I, M Harinath/192425020 (Name/Reg No) certify that **this** submission is my original work and that I have adhered to **the** guidelines specified for **this assessment**.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q. No

Understanding of
Concept (20)

Experimental
Design (20)

Use of Tools

C03 AT3

Co3

Syntax Directed Translation and Attribute Dependenci

4ses

a Syntax

directed

engin

e

Translati

on

LR bottom-up

Parser.

dynthesized and inherited

and

The Compiler

engine

Scheme

Scheme with

are

with attributy

The
evaluated
Problem is that inherited attributes after
reductions, resulting

main in Circular
Code
dependencies, repeated evaluation
and in Complete

1. Attribute Analysis

$e \rightarrow e + T$

$e \rightarrow T$

$T \rightarrow T * F$

$\rightarrow F \rightarrow (e) \rightarrow F$

Let C.place, T.place and F.place be
synthesized

attributes

$e \rightarrow E, + T$

Semantic rule is

e.place - new temp

()

emit (e.place - C1-place + T.
place)

T- TIX F

T. place

new temp

()

t

-emit (T-place - Ti- place
*F. place)

Dependency
Graph

The

A.in B. Syn

B. Syn

→ Cin

C.in ->A.in

→

A.syn A.in

given

attribute

d

definition is neither -attributed *no*

Cies?
tion

Corrected

Translation Scheme di tama
input o
(01

GR passer



Semantic Stack



Synthesized
Attributes



Three address Code.

bnowego

-This Preserves operator precedence. Support
neated paranthesia

intermidiate code exdiciently
during LR parsing.

and

genera

tes Synt ax

Tree and Common Pub expression
elimination

The

is

given

expression (a+b)

+ (a+b) ~ (a+b),

unoptimized TAC

t0 = a+b

t1 = a+b

193 t2 = t0 *t,

tz = a+b

もろ

ty = t2 -

tz

AT

optimized

Taciue

q
900

bow

D

ولودية

Since atb is Commen,
Calculate it only

Tidmo t1 = a+boor
ban

18 bot2 = t1*t,

ts: tr- ts

tbilgi

*100

Computational

Reduction

Reduction .

$$- [(5 - 3) / 5] \cdot$$

4.

$$(2/5) \times 100$$

hubut?

2 40%.

Avitlaustre reduction =

40%. Avithamptée

Jof: bi

لمره

الله

onci

6

Correct Subtree Comparison

Jany

For example, `int (a) + int (b)`
and `float(&) + Float (b)`

The Tree `nod.` Should

Contain

operator Left child

Right Child

operand

Type

Result Type

bits Conversion

information

Debugging

Strategy

c

storbicoteri

The optimizer should

verity: bow

1. Same operator 2. Same

Operands

operand

order

From

5. Same result
type

3. Same

4. Same

Operand

type

s

Arithmetic

operators

6. Game Conversion
information.

the optimized TAC reduces
arithmetic

5 to 5,

giving

a 40 %
reduction..

and Type Gauivalence.

basicaitoo Type Checking and

Type Gauivalence

Type Check

Compiler

Supports

The Compiler Supports

integers, floating point

values, Characters, arrays and records.

The Current type

Che CKA

in

n Correctly mixes name

exviivalence and structural earivalence

and performs implicit Conversation

Checking Compatibility

before

lub Loboto

(21(2)

Name

Gavvivalence (2)

type Student =

record

id:int

maus Float

044

type employee to

record.. id: int

marks: Float

end

Away

and

Record

Checki

ng

```
1 int A [10]
   float B [10]
```

then:

```
"Type [A] = array
(10, int) -Type (B) =
array (10, float)
[B]=
```

Hence

Type(A) +
Type (B)

For records

Student s

Employ
ee

e

Student &
Employee

JAT bosimilgo

it

Type Compatibility must be
checked before

Conversion

n widening

Can

gener
ally

Conversions such as int - float
be allowed
automatically.

4

(A) vid

Three - Address Code and

The

$x =$
given expression

give

n

Register
allocation.

$(a+b) / (cxd)$

unoptimized

Translation.

$t1 = a$

$t2 = b$

$tz = t1 * t2$

$ty = c$

=

$to = tu * ts \quad t7$

$= t3/t6$

$X = tz$

optimized TAC

$t1 = a * b$

$t2 = c * d$

$t1 / t2$

$2 = t3$

Register

Allocation

Assign $t2 \rightarrow$

Rr

: $t1 \rightarrow$

R1

18

brajs

Abian

Jobute

palam9

trebut.

toolt

Moy R., a wildingmo

MUL 12, b

MOV R2, C

MUL
R2, d

DIV
R1, R2

120V x, R,

Register

Lifetime

After, Rc a *b

the operands a and b

Live (a) - FALSE

Live (b) =

FALSE

was bio

are no

longer
required

Attribute evaluation and
registers

During
bottom up

Coun

Contain

a

e. place =

R1

Ti place = R2

instead of

registers wall

07 translation each

synthetize

place field.

C. place = new
temp ()
every time. the
Compiler checks

synthudized
attribute

1&
a
register
available?
18 its previous
Nalue dead?
Can it be recused:
Comparisi
on
Parameter

Temporaries

Register

Mov

Before

After

Excessive batidas

Reduced

H

reuse

No

Yes

many

y

Tugri

Low

instructions

Execution efficiency

The Compiler
uses a

FOLLO

W

Jew

Highe
r.

C predictive passing, first!
FoLLiow and inherited
attributes uses a predictive parser
and semantic analyzes. First and Follow
sets are used for production selection,
while inherited attributes propagate
type

•information

1 . First set

$A \rightarrow id$

$FIRST(A) =$

$\{id\}$

For AE

we have $\& G FIRST(A)$

2. Follow det.

For $S \rightarrow a$:

1 $C \text{ FOLLOW } (A)$

3. Epsilon production

$A \rightarrow id$

$A \rightarrow \epsilon$

Then, $FIRST(id) =$

$\{id\}$

it $FOLLOW(a) = \{ ;$
 $\}$

$= \{ : \}$

The

Parser uses

lookahead = id

But

– $A \rightarrow id$

lookahead = ;

$\leftarrow A \rightarrow M$

4. Error in inherited attribute propagation

Correct older

didte balvador

input

ال

011

FIRST FOLLOW
decision)

production
selection:

↓

ri**bb**, notw

inherited attribute
propagation

Semantic
action

Symbol

- 1

-table update

tib

b- attributed Definition

for :

the type

Can be
propagated

bin=

T

type

17

T.

Type

type

= int

An

then, L.in = int

as:

t-attributed definition is sufficient for this
left-to-right propagation, provided
that semantic

executed only after the
correct production

actions

are

is selected.

to

Select

The parser must first use

the

production

HH

first and

Rollow